ETAPA 2 - CALCULE AS ESTIMATIVAS

Operações de gravação: 100 milhões de URLs por dia = **100.000.000 / 24 / 60 / 60 = 1160 RPS**

Operações de leitura: 10:1 = **1160 * 10 = 11.600 RPS**

Tempo de armazenamento das URLs: 10 anos = **100.000.000 * 365 * 10 = 365 Bilhões de registros**

Capacidade de armazenamento: 100 bytes por URL = **365 Bilhões * 100 bytes = 36,5Tb**

ENDPOINTS

```
// Endpoint de encurtamento da URL

*Request:
 POST api/v1/shorten
 Body: {"url": "www.example.com..."}

*Response:
 Status Code: 201 (created)
 {"short_url": "bit.ly/zn9e10A"}
```

```
// Endpoint de redirecionamento da URL

*Request:
 GET "bit.ly/zn9e10A"

*Response:
 Status Code: 301/302 (redirect)
 Header:
    Location: "www.example.com/..."
```

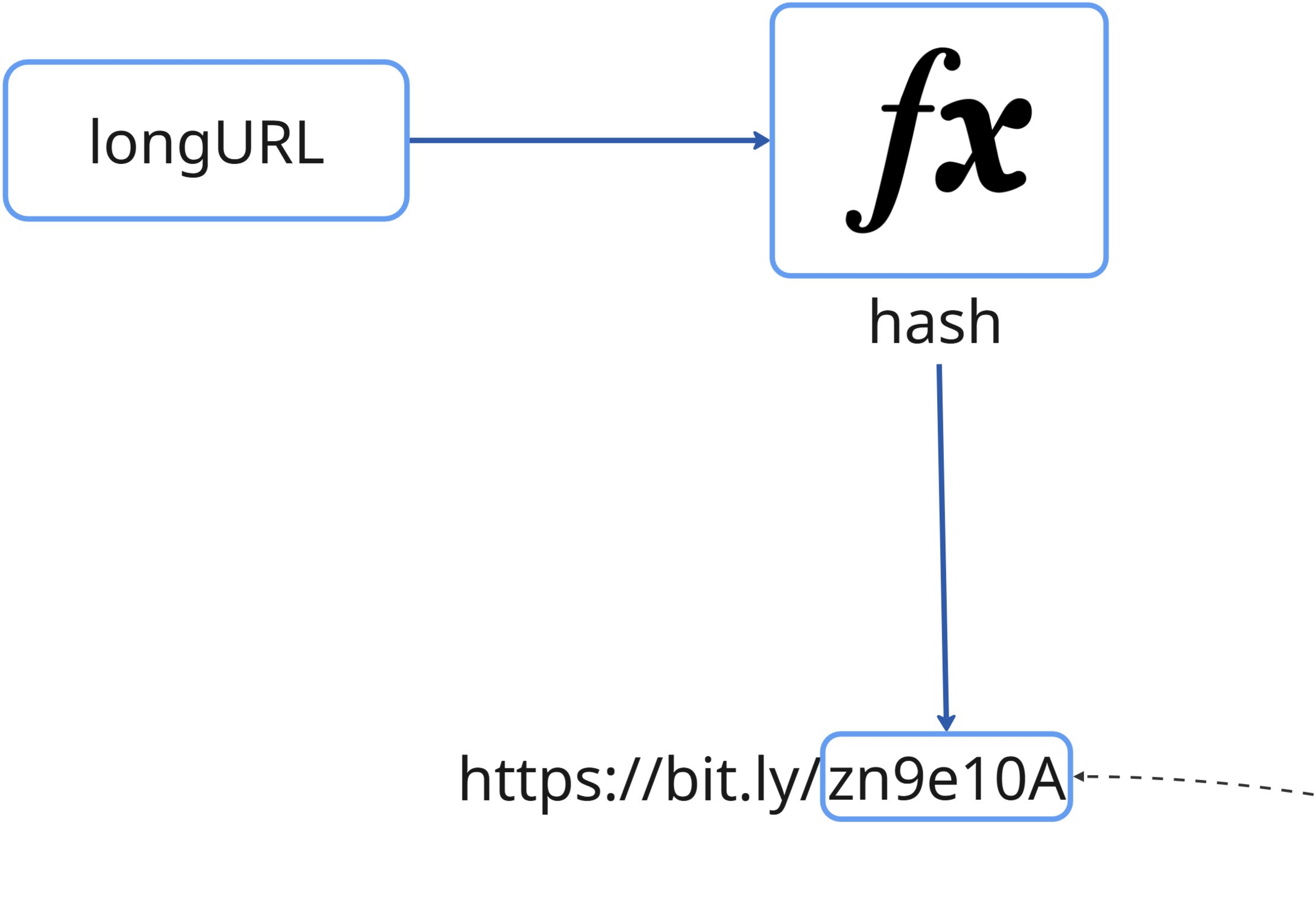
CQL - Cassandra Query Language

Apache CASSANDRA

```
╔═══════════════════════════════════════════╗
║              Table: url                   ║
╠═══════════════════════════════════════════╣
║ PRIMARY KEY: shortcode                    ║
║ PARTITION KEY: shortcode                  ║
╟───────────────────────────────────────────╢
║ Columns:                                  ║
║   • long_url (text)                       ║
║   • created_at (timestamp)                ║
╚═══════════════════════════════════════════╝


CREATE TABLE url (
    shortcode TEXT PRIMARY KEY,
    long_url TEXT,
    created_at TIMESTAMP
);
```

New grid

shortcode	long_url	created_at
zn9e10A	https://en.wikipedia.org/wiki/Systems_design	2025-10-23T18:42:15.123Z


longURL
fx
hash
https://bit.ly/zn9e10A

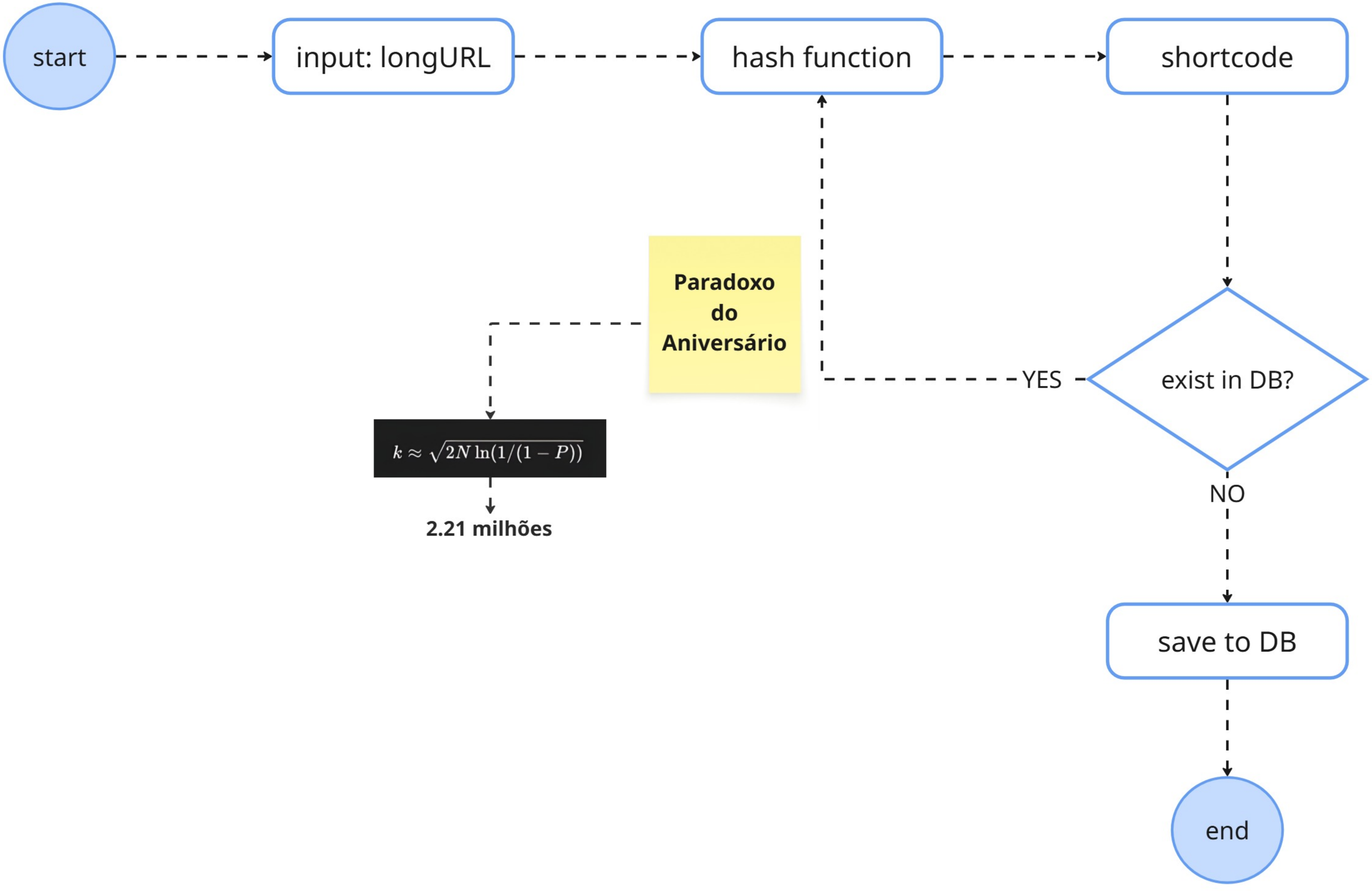
ETAPA 3 - CALCULE O HASH

10 dígitos numéricos: 0 - 9
26 letras minúsculas: a - z
26 letras maiúsculas: A - Z
Total: 10 + 26 + 26 = 62 caracteres

0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

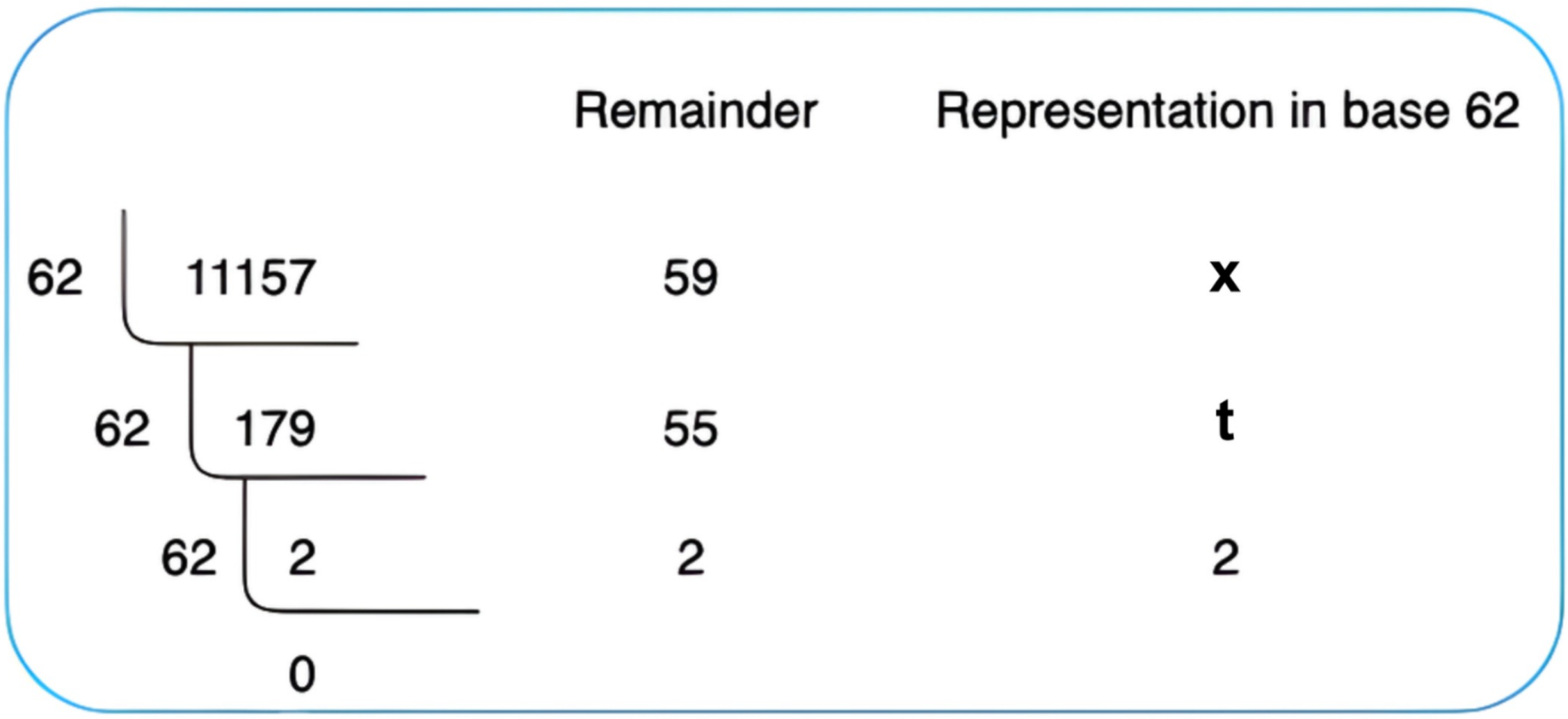
n	Número máximo de URLs
1	62^1 = 62
2	62^2 = 3.844
3	62^3 = 238.328
4	62^ 4 = 14.776.336
5	62^5 = 916.132.832
6	62^6 = 56.800.235.584
7	**62^7 = 3.521.614.606.208 = ~3,5 trilhões**
8	62^8 = 218.340.105.584.896

COLISÃO DE HASH



CONVERSÃO DE BASE 62

Decimal	Binary	Base62	Decimal	Binary	Base62	Decimal	Binary	Base62	Decimal	Binary	Base62
0	000000	0	16	010000	G	32	100000	W	48	110000	m
1	000001	1	17	010001	H	33	100001	X	49	110001	n
2	000010	2	18	010010	I	34	100010	Y	50	110010	o
3	000011	3	19	010011	J	35	100011	Z	51	110011	p
4	000100	4	20	010100	K	36	100100	a	52	110100	q
5	000101	5	21	010101	L	37	100101	b	53	110101	r
6	000110	6	22	010110	M	38	100110	c	54	110110	s
7	000111	7	23	010111	N	39	100111	d	55	110111	t
8	001000	8	24	011000	O	40	101000	e	56	111000	u
9	001001	9	25	011001	P	41	101001	f	57	111001	v
10	001010	A	26	011010	Q	42	101010	g	58	111010	w
11	001011	B	27	011011	R	43	101011	h	59	111011	x
12	001100	C	28	011100	S	44	101100	i	60	111100	y
13	001101	D	29	011101	T	45	101101	j	61	111101	z
14	001110	E	30	011110	U	46	101110	k			
15	001111	F	31	011111	V	47	101111	l			

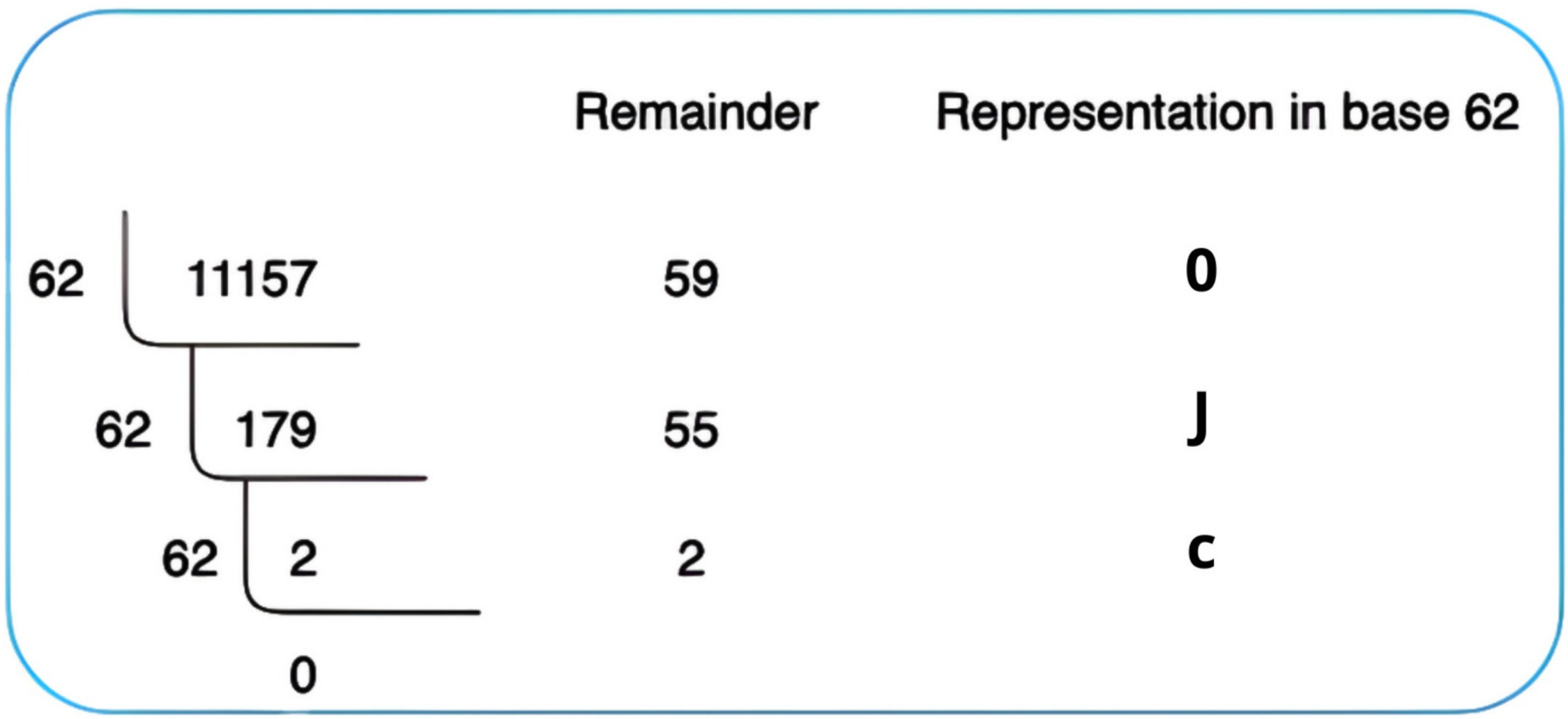


https://bit.ly/2tx

```python
import base62

number = 11157

encoded = base62.encode(number)
print("Base62:", encoded) // 2tx

decoded = base62.decode(encoded)
print("Decimal:", decoded) // 11157
```

BASE 62 + OFUSCAÇÃO COM HASHID

Decimal	Binary	Base62	Decimal	Binary	Base62	Decimal	Binary	Base62	Decimal	Binary	Base62
0	000000	I	16	010000	u	32	100000	m	48	110000	A
1	000001	t	17	010001	K	33	100001	W	49	110001	R
2	000010	c	18	010010	D	34	100010	z	50	110010	a
3	000011	y	19	010011	i	35	100011	X	51	110011	H
4	000100	8	20	010100	T	36	100100	5	52	110100	C
5	000101	3	21	010101	O	37	100101	M	53	110101	N
6	000110	F	22	010110	E	38	100110	f	54	110110	s
7	000111	b	23	010111	x	39	100111	h	55	110111	J
8	001000	L	24	011000	P	40	101000	d	56	111000	p
9	001001	j	25	011001	g	41	101001	B	57	111001	w
10	001010	1	26	011010	Y	42	101010	e	58	111010	q
11	001011	4	27	011011	U	43	101011	Z	59	111011	0
12	001100	S	28	011100	2	44	101100	G	60	111100	k
13	001101	V	29	011101	l	45	101101	n	61	111101	Q
14	001110	v	30	011110	6	46	101110	7			
15	001111	o	31	011111	9	47	101111	r			



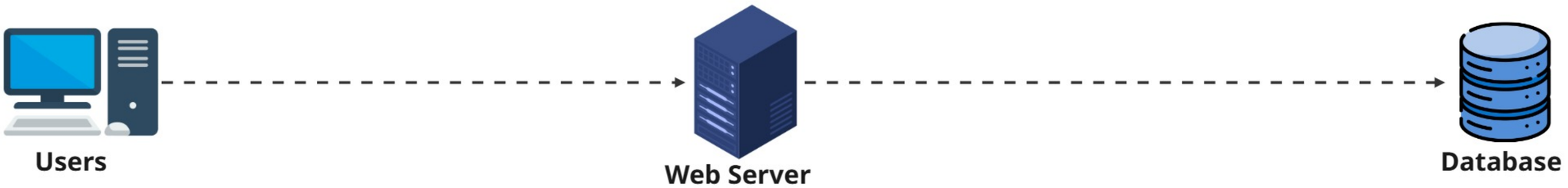
https://bit.ly/cJ0

FUNÇÕES DE HASH

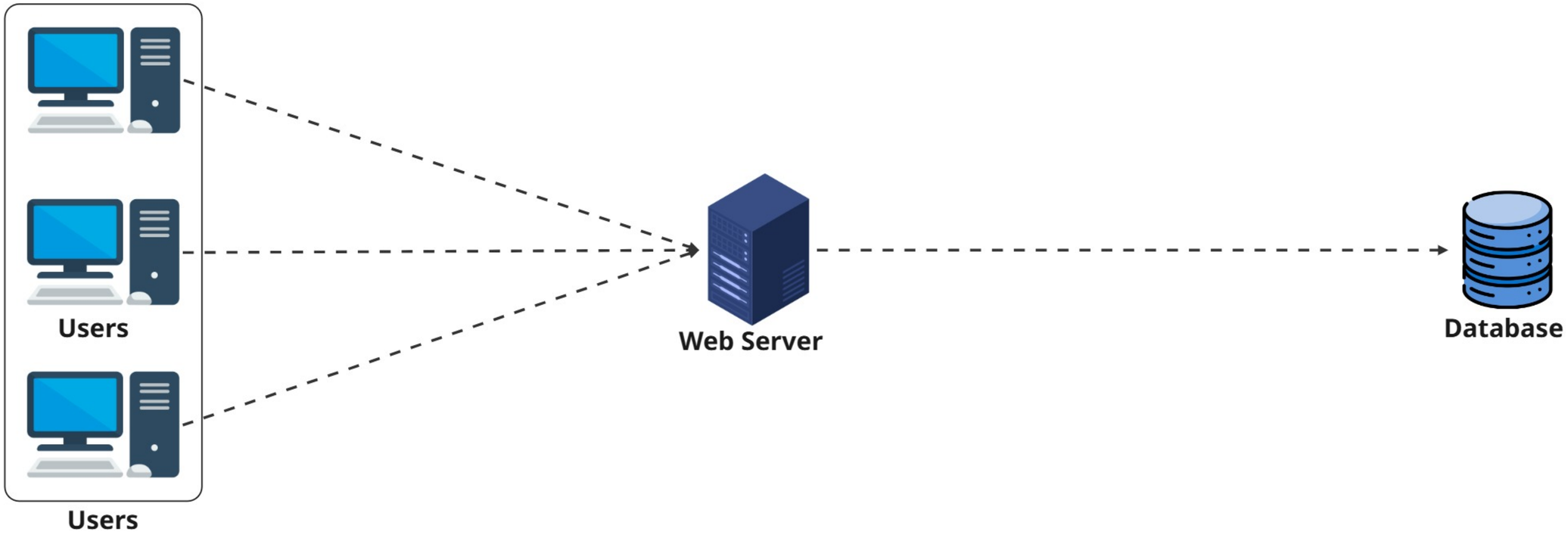
Função hash	Valor de hash (hexadecimal)
CRC32	5cb54054
MD5	5a62509a84df9ee03fe1230b9df8b84e
SHA-1	0eeae7916c06853901d9ccbefbfcaf4de57ed85b

```php
$longUrl = "https://www.example.com/...";
$shortCode = md5($longUrl);

print $shortCode; //b7c1a9a3c42b6d6e36a7e4c0a9f474b4b63d1d9b

print substr($shortCode, 0, 7); //b7c1a9a
```

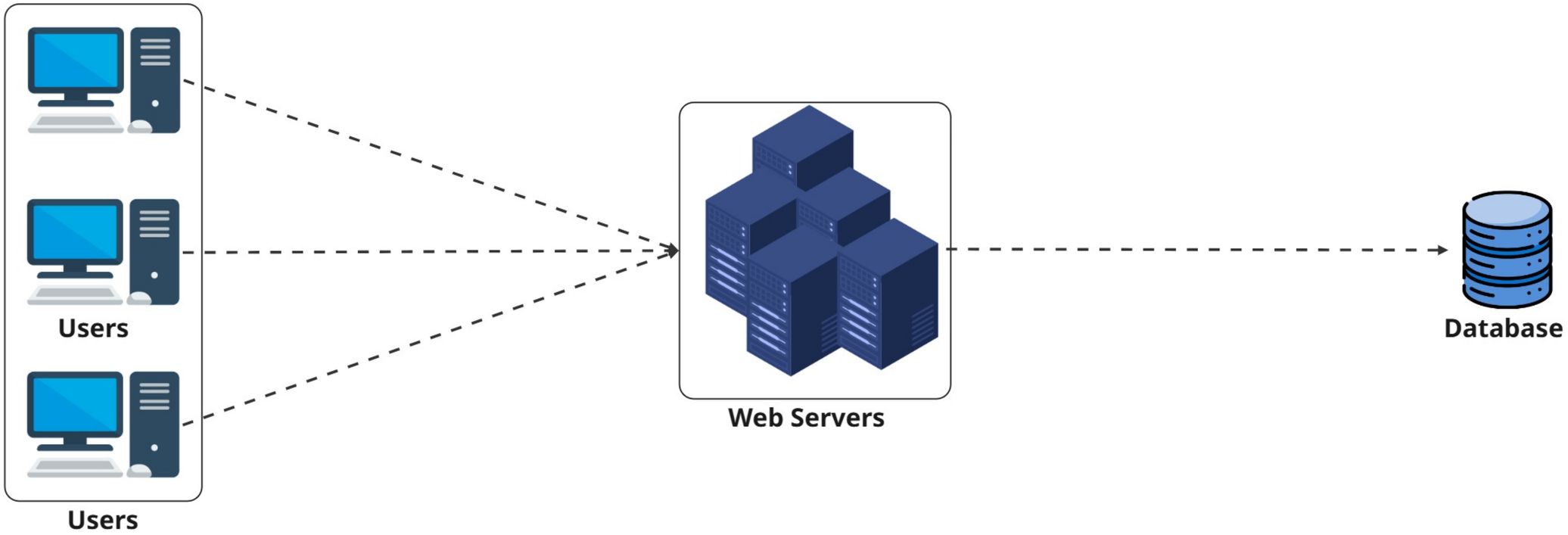
SYSTEM DESIGN



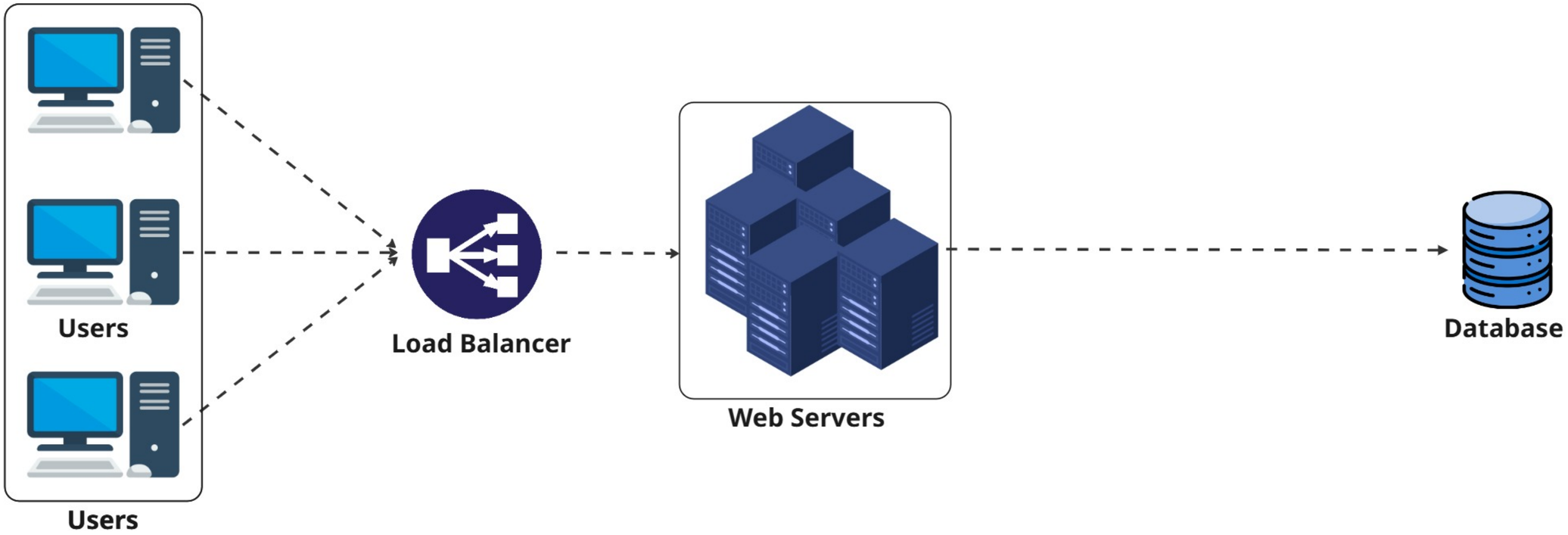
SYSTEM DESIGN



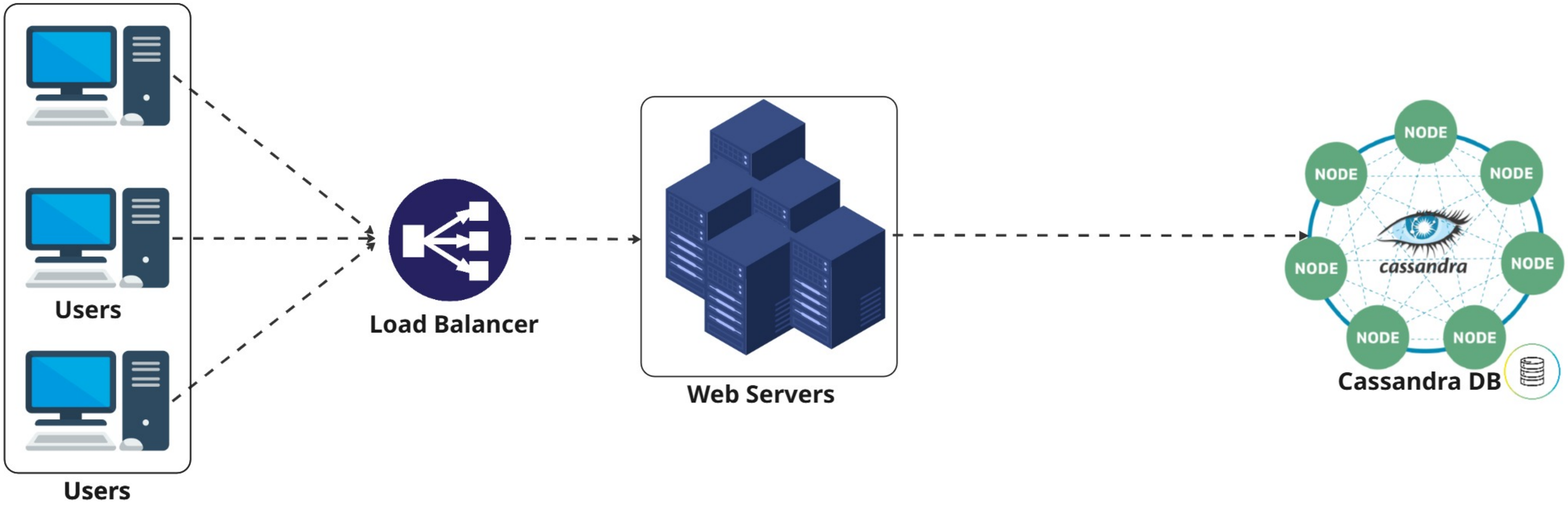
SYSTEM DESIGN



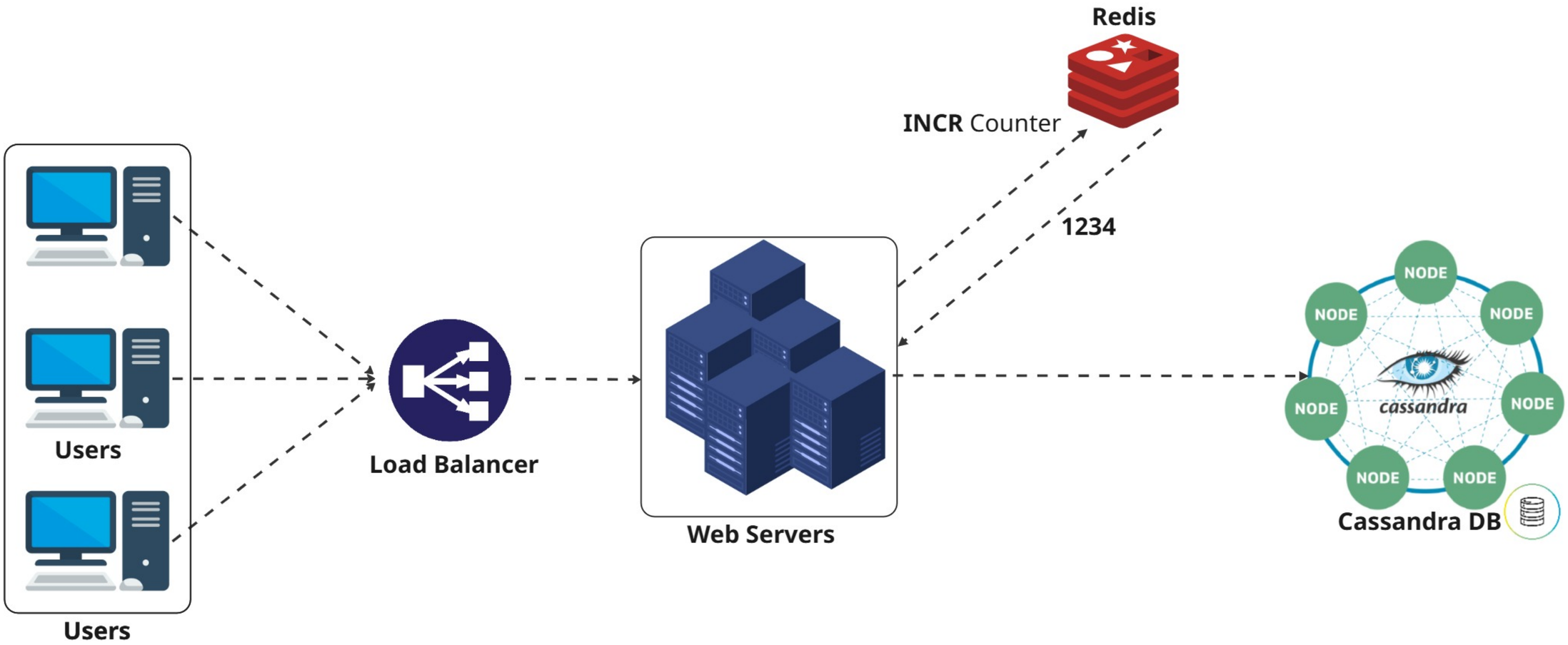
SYSTEM DESIGN



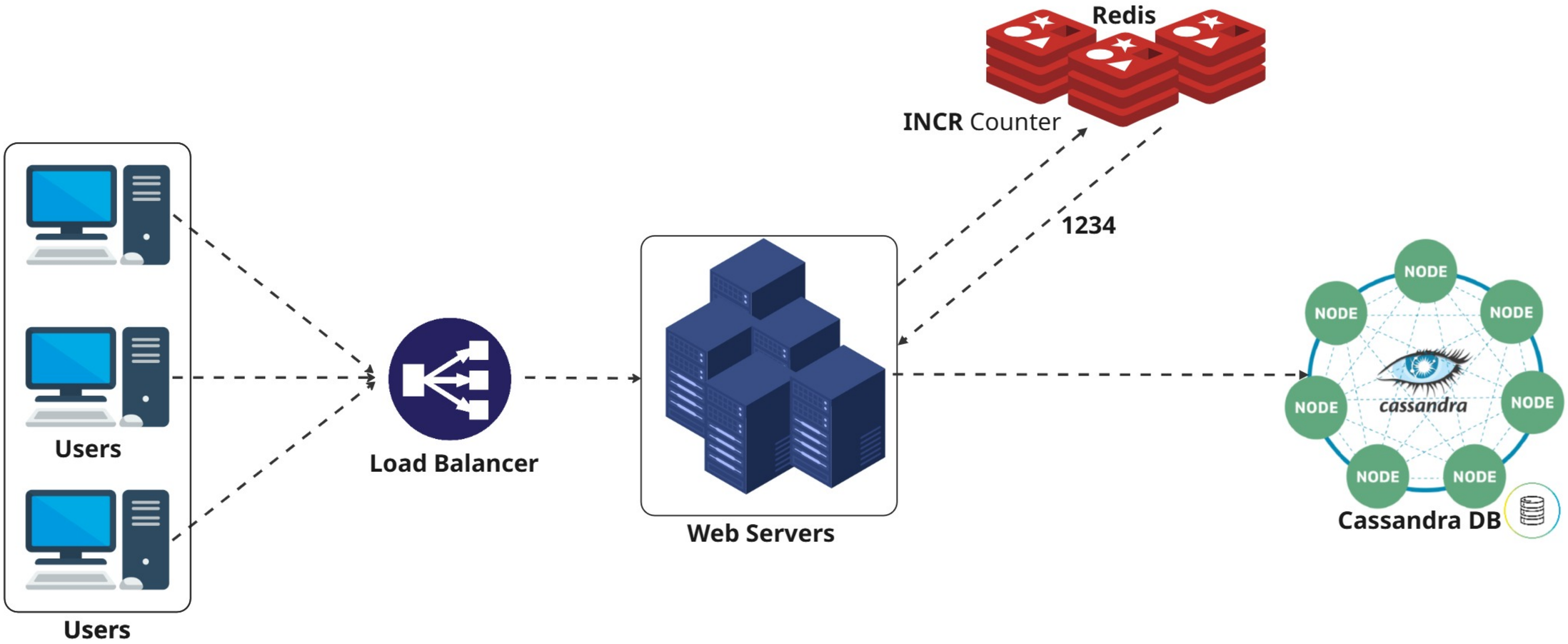
SYSTEM DESIGN



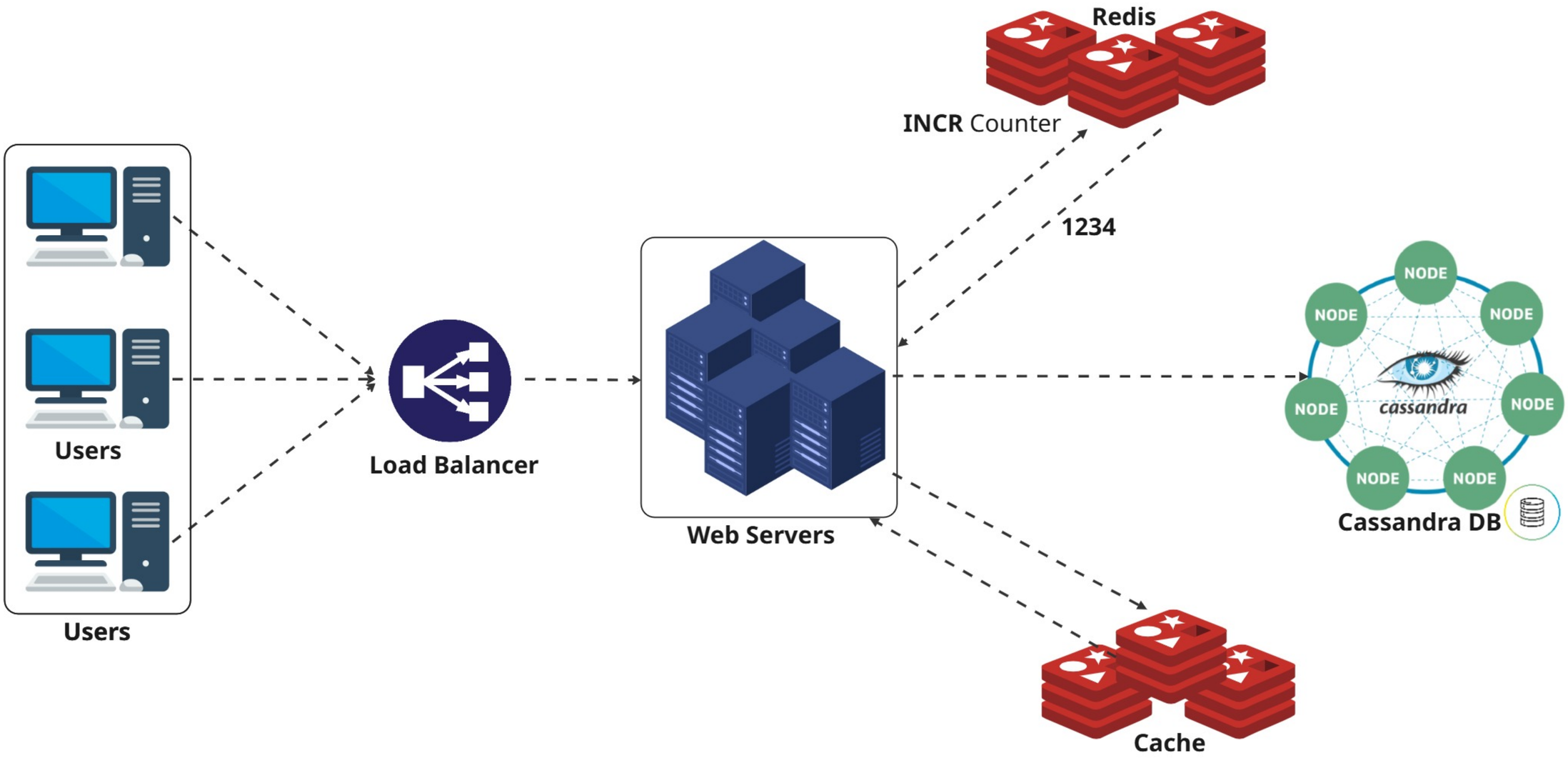
SYSTEM DESIGN



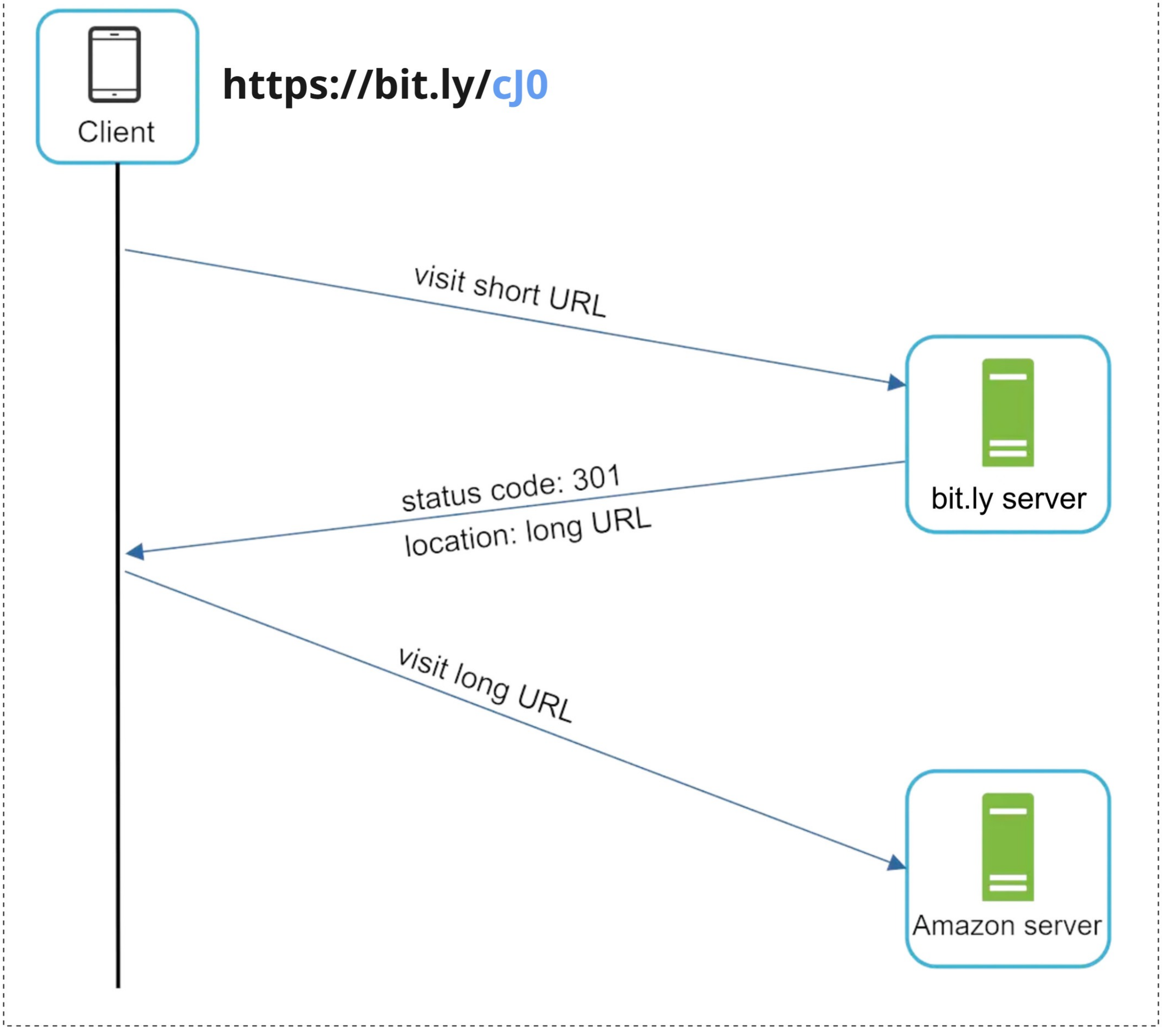
SYSTEM DESIGN



SYSTEM DESIGN



REDIRECT URL



ETAPA 1 - ENTENDA O PROBLEMA E ESTABELEÇA OS REQUISITOS

REQUISITOS FUNCIONAIS

1 - Encurtamento de URL: dado um URL longo => retornar um URL muito mais curto

2 - Redirecionamento de URL: dado um URL mais curto => redirecionar para o URL original

REQUISITOS NÃO FUNCIONAIS

1 - O sistema deve suportar 100 milhões de URLs geradas por dia.

2 - O tamanho da URL encurtada deve ser o mais curto possível.

3 - Somente Números (0-9) e caracteres (az, AZ) são permitidos na URL

4 - Para cada 1 operação de gravação no banco de dados, haverão 10 operações de leitura

5 - O comprimento médio das URLs armazenadas é de 100 bytes

6 - URLs devem ser armazenadas pelo período mínimo de 10 anos

7 - O sistema deve operar em modo de alta disponibilidade (24/7)

Toda arquitetura é feita para atender requisitos muito bem definidos